INF384 · TEMAS AVANZADOS EN INGENIERÍA DE SOFTWARE 1 · 2026-2

Sesión 10

Automatización de seguridad: infraestructura, ejecución y política

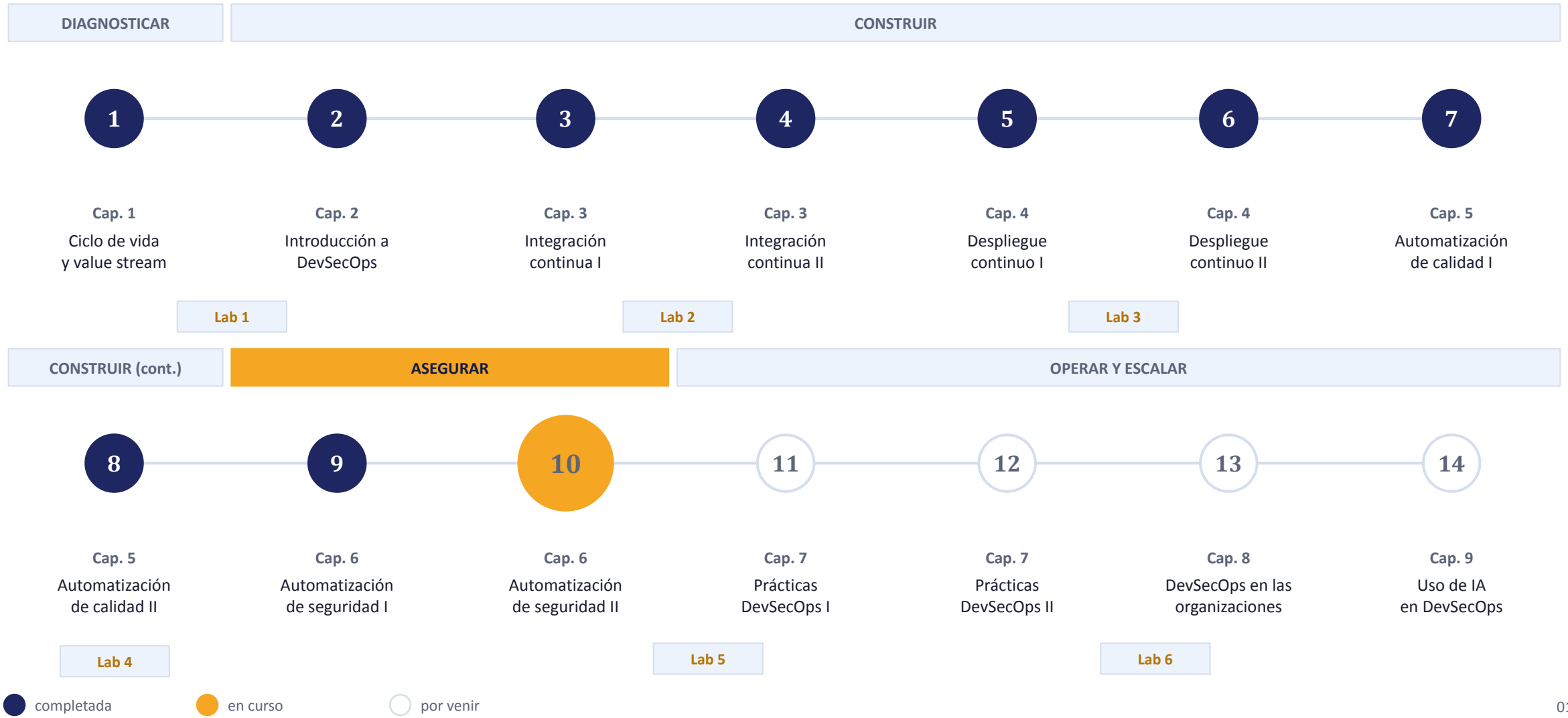
Policy as code sobre la infraestructura declarada, análisis dinámico sobre lo desplegado y el mapa de controles del pipeline

---

Capítulo 6 (2 de 2) · 3 horas | RA3 · RA4 | Prof. José Felipe del Castillo Salazar

INF384 · 9 CAPÍTULOS · 14 SESIONES · 42 HORAS

La ruta del curso



SESIÓN 10

Objetivo de la sesión y ruta

1. **Policy as code sobre la infraestructura declarada**
2. **Análisis dinámico de la aplicación desplegada**
3. **La política completa: mapa de controles y gestión de hallazgos**

A. **Actividad: políticas y análisis dinámico para su caso**

L. **Laboratorio 5**

Resultados de aprendizaje vinculados: RA3 (seguridad integrada con análisis de IaC y dinámico) y RA4 (impacto en la confiabilidad de lo desplegado).

CONTINUIDAD CON LA SESIÓN 9

Dónde estamos en el pipeline

Commit	Build	Pruebas	Calidad	**Seguridad**	Publicar	**Desplegar**	**Observar**

Lo que la sesión 9 dejó resuelto

- El analizador estático sigue el dato de la fuente al sumidero; el de composición lee el lock file.
- La prioridad combina alcanzabilidad, exposición, explotación y parche.
- La imagen se analiza antes de publicar, en el registro y al desplegar; firma e inventario viajan con el digest.
- Lo que entra al runner con credenciales se fija por digest.

Lo que todavía no está decidido

- ¿Cómo se revisa un bucket público o un rol con permisos totales antes de que existan?
- ¿Qué defecto solo aparece con la aplicación corriendo, y en qué etapa del pipeline se busca?
- ¿Cómo se ve la política completa de un pipeline, con todos los controles y lo que decide cada uno?
- ¿Qué hace el equipo con 200 hallazgos de cinco herramientas distintas?

Hoy la seguridad se extiende a la derecha del pipeline: a la infraestructura que se va a crear y a la aplicación que ya responde.

A

Policy as code sobre la infraestructura

La configuración insegura como vulnerabilidad, y la regla escrita como código que la detiene antes de aplicar

La configuración insegura es una vulnerabilidad sin código

Recurso declarado	Configuración frecuente	Consecuencia
Almacenamiento de objetos	Acceso público de lectura para servir imágenes	Cualquier archivo del bucket es descargable, incluidos respaldos
Grupo de seguridad	Puerto de administración abierto a 0.0.0.0/0 'para probar'	Acceso directo desde internet al servidor
Rol de ejecución	Política con acción * sobre recurso *	Una falla en la aplicación se convierte en control total de la cuenta
Base de datos	Sin cifrado en reposo ni respaldos automáticos	Exposición de datos y pérdida sin recuperación
Función serverless	Variables de entorno con credenciales en texto plano	Las credenciales quedan visibles para cualquiera con acceso de lectura

1

Ningún analizador de la sesión 9 ve esto

El código de la aplicación es correcto; la infraestructura que lo ejecuta es la que expone los datos.

2

La infraestructura como código lo hace revisable antes de que exista

Un archivo declarado se analiza igual que el código fuente: en el commit, con una regla y una decisión.

Tres momentos para evaluar la infraestructura



	1 · Sobre el código	2 · Sobre el plan	3 · Sobre lo que existe
Qué lee	Los archivos declarados, con variables sin resolver	El plan resuelto: valores finales y acciones exactas	La configuración viva consultada al proveedor
Qué detecta	Patrones en la declaración: cifrado ausente, acceso público	Lo que las variables ocultaban y las destrucciones	Drift: lo que cambió por consola o fuera del pipeline
Qué se le escapa	Un valor que llega por variable de entorno o módulo	Nada de lo declarado; sí lo creado fuera del código	Lo que nunca se declaró y lo que va a cambiar mañana
Cuándo corre	En cada commit, segundos, sin credenciales	En cada PR, con el plan, sin aplicar nada	Programado: diario o continuo
Quién decide	El gate del pipeline	El gate del pipeline, con el plan comentado	Alerta a operación; corrección por pipeline

El Laboratorio 3 evaluó el momento 1. El Laboratorio 5 agrega el momento 2: la misma regla sobre el plan ve los valores que la declaración no mostraba.

Anatomía de una política escrita como código

POLITICAS/ALMACENAMIENTO_PUBLICO.REGO — SOBRE EL PLAN

```
package infra.almacenamiento                              # 1

import input.resource_changes as cambios                  # 2

deny[msg] {                                               # 3
  r := cambios[_]
  r.type == "aws_s3_bucket_public_access_block"
  r.change.after.block_public_acls == false               # 4
  msg := sprintf("%s permite ACL publica; " +
    "excepcion junto al recurso", [r.address])            # 5
}

warn[msg] {                                               # 6
  r := cambios[_]
  r.type == "aws_lambda_function"
  not r.change.after.tracing_config
  msg := sprintf("%s sin trazas activas", [r.address])
}
```

1. **La política vive en el repositorio**
   Se versiona, se revisa en PR y tiene historial igual que el código.

2. **La entrada es el plan en JSON**
   La regla ve valores resueltos y acciones: create, update, delete.

3. **deny detiene; warn informa**
   La misma distinción de la S4, escrita en la regla.

4. **La condición es explícita y legible**
   Quien revisa el PR entiende qué se exige sin abrir la herramienta.

5. **El mensaje dice qué hacer**
   Nombra el recurso y la vía de excepción; la excepción tiene cuatro datos.

6. **Lo informativo también queda escrito**
   Observabilidad y etiquetas entran como warn hasta que el equipo acuerde subirlas.

Qué políticas escribir primero

Exposición	Identidad	Datos	Trazabilidad
Acceso público, puertos abiertos a internet, endpoints sin autenticación.	Comodines en acciones o recursos, roles compartidos entre funciones, claves de larga duración.	Cifrado en reposo y en tránsito, respaldos, retención, borrado protegido.	Registros de acceso, trazas, etiquetas de dueño y ambiente en cada recurso.
`deny`	`deny`	`deny`	`warn → deny`

1. **Primero lo que ya fue incidente y lo que el regulador pide por escrito**
   Un conjunto inicial de diez a veinte reglas que todos entienden vale más que trescientas que nadie lee.
2. **La regla nueva entra en warn y sube a deny por acuerdo**
   Igual que el umbral de la sesión 3: se mide cuántos recursos la incumplen hoy antes de detener a alguien.
3. **Toda excepción vive junto al recurso, con vencimiento**
   Al vencer, la política vuelve a detener sola. El número de excepciones vigentes indica si la regla está bien calibrada.

Cuando lo que existe dejó de ser lo declarado

Cómo aparece

Un cambio por consola durante un incidente a las 2 a. m.: se abre un puerto, se amplía un rol, se desactiva una alarma. El incidente se resuelve; el cambio se queda. El archivo declarado sigue diciendo lo de antes.

Cómo se detecta

Un plan programado sin cambios de código: si propone acciones, hay drift. Y la misma política de la slide 07 evaluada sobre la configuración viva, de forma continua, con alerta en lugar de gate.

1

La corrección entra por el pipeline, nunca por consola

Si el cambio de emergencia era necesario, se declara en el código y se aplica; si no, el apply lo revierte.

2

El cambio de emergencia tiene una ruta escrita

Quién puede hacerlo, cómo se registra y en cuánto tiempo se vuelve a declarar. Es la aprobación de la sesión 6 para el camino de vuelta.

El pipeline garantiza lo que pasa por él. Lo que entra por otra vía solo se detecta comparando lo declarado con lo que existe.

B

Análisis dinámico

Lo que solo se ve con la aplicación respondiendo, y en qué etapa del pipeline se busca

Lo que el análisis dinámico ve y el estático no

Aspecto	Análisis estático	Análisis dinámico
Qué necesita	El código fuente	La aplicación desplegada y una dirección
Qué observa	Caminos posibles en el código	Respuestas reales a peticiones reales
Qué detecta mejor	Flujos fuente-sumidero, secretos, patrones	Cabeceras y cookies, TLS, configuración del servidor, autenticación, componentes integrados
Qué se le escapa	Configuración en ejecución, servidor, red	Código que ninguna petición alcanza; la causa exacta en el código
Falsos positivos	Frecuentes sin calibrar	Pocos: el hallazgo viene con la petición y la respuesta que lo prueban
Cuándo corre	En cada commit, minutos	Después de desplegar en certificación; minutos en modo pasivo, horas en activo

Un hallazgo dinámico trae la petición que lo provocó y la respuesta que lo confirma. Es la evidencia más directa que produce cualquier control del capítulo.

Dos modos: observar y atacar

Pasivo (línea base)

- Recorre la aplicación con peticiones normales y observa las respuestas.
- Detecta cabeceras ausentes, cookies inseguras, información filtrada en errores, versiones expuestas.
- Minutos; sin efecto sobre datos; corre en cada despliegue de certificación.

Activo

- Envía peticiones diseñadas para provocar fallas: inyección, recorrido de rutas, manipulación de parámetros.
- Detecta lo explotable desde fuera, con la prueba adjunta.
- Horas; crea y modifica datos; corre programado contra certificación, nunca contra producción sin acuerdo.

1. **Sin autenticación, el análisis ve solo la página de ingreso**
   Se le entrega una sesión de prueba o un token de un usuario de prueba, con los datos de la sesión 8: propios y limpiados al final.

2. **Una API sin interfaz se recorre con su especificación**
   El documento de la API es el mapa: cada operación declarada se prueba con sus parámetros y tipos.

3. **El umbral se declara igual que en cualquier gate**
   Un hallazgo alto en línea base detiene la promoción; lo demás se registra con responsable.

CAPÍTULO 6.5 · UBICACIÓN

El análisis dinámico en la secuencia del pipeline

Integración continua	Publicar	Certificación	Programado	Producción
Secretos, estático, composición, imagen, política sobre el código y el plan.	Imagen firmada con inventario, por digest.	Despliegue, humo, end to end, análisis dinámico pasivo en cada despliegue.	Análisis dinámico activo nocturno sobre certificación; evaluación de postura sobre lo vivo.	Humo, observación, verificación sintética; sin análisis activo.

GITHUB ACTIONS — JOB DINÁMICO EN CERTIFICACIÓN

```
dast-base:
  needs: desplegar-cert                                  # 1
  steps:
    - uses: <analizador-dinamico>/base@<sha>
      with:
        objetivo: ${{ needs.desplegar-cert.outputs.url }}
        especificacion: openapi.yaml                     # 2
        umbral: alto                                     # 3
    - uses: actions/upload-artifact@<sha>
      if: always()                                       # 4
      with: { name: dast-base, path: reporte/ }
```

1. **Recibe la dirección del despliegue**
2. **La especificación guía el recorrido**
3. **El umbral decide la promoción**
4. **El reporte se conserva aunque falle**

C

La política completa

El mapa de controles del pipeline, un solo flujo de hallazgos y el marco de referencia

El mapa de controles del pipeline

Control	Etapa	Efecto	Umbral de ejemplo	Evidencia que produce
Detección de secretos	CI · cada commit	**Detiene**	Cualquier hallazgo	Hallazgo con archivo y commit
Análisis estático (SAST)	CI · cada commit	**Detiene**	Alto en reglas de alta confianza	Camino fuente-sumidero
Composición (SCA)	CI · cada commit	**Detiene**	Alto con parche disponible	Camino hasta la directa; inventario
Cadena de suministro	CI · revisión de PR	**Detiene**	Acción o base sin digest	Diff del workflow y del Dockerfile
Análisis de imagen	CI · antes del push	**Detiene**	Alto en SO o runtime con parche	Reporte por capa; firma
Política sobre el código IaC	CI · cada commit	**Detiene**	Exposición, identidad, datos	Regla y recurso
Política sobre el plan	CI · cada PR	**Detiene**	deny de la política; destrucciones	Plan comentado en el PR
Dinámico pasivo	Certificación · cada despliegue	**Detiene**	Alto	Petición y respuesta
Dinámico activo	Programado · nocturno	**Informa**	Alto ⟶ ticket con plazo	Petición y respuesta
Postura sobre lo vivo	Programado · continuo	**Informa**	Drift o deny sobre lo existente	Recurso y diferencia

Ocho controles detienen y dos informan. Cada fila tiene su umbral escrito y su evidencia; con eso la política completa cabe en una página del repositorio.

Diez herramientas, un solo flujo de hallazgos



1. **Todos los reportes entran en un formato común a un solo lugar**
   El mismo componente vulnerable aparece en composición y en imagen: es un hallazgo, con dos fuentes.

2. **El triage aplica las cuatro preguntas de la sesión 9**
   Alcanzable, expuesto, explotado, con parche. Lo hace el equipo que escribió el código, con el criterio escrito.

3. **La decisión es una de tres y queda registrada**
   Corregir con fecha, exceptuar con cuatro datos, o aceptar el riesgo con la firma de quien responde por él.

4. **Las métricas del flujo son tres**
   Tiempo de corrección por severidad, excepciones vigentes, y porcentaje de hallazgos con decisión. Un backlog creciente sin decisión es el síntoma de que el umbral está mal.

El flujo de hallazgos es un flujo de trabajo más del equipo, en su misma herramienta y con su misma cadencia. Un tablero aparte del área de seguridad se convierte en una lista que nadie abre.

Los controles del capítulo en el marco de referencia

Práctica	Qué pide	Control del pipeline que la cubre
PO.3	Implementar y mantener la cadena de herramientas de forma segura	Acciones y base fijadas por digest; runner con instalación sin scripts
PW.4	Reutilizar software existente de forma segura	Composición sobre el lock file; inventario (SBOM) con cada digest
PW.6	Configurar el proceso de compilación para mejorar la seguridad	Build en CI que solo produce imagen si las compuertas pasan; firma
PW.7	Revisar y analizar el código legible por humanos	Análisis estático; detección de secretos; política sobre el código IaC
PW.8	Probar el código ejecutable	Pruebas del Capítulo 5; análisis dinámico en certificación
PW.9	Configurar el software con valores seguros por defecto	Política sobre el plan; imagen base reducida
RV.1 · RV.2	Identificar vulnerabilidades de forma continua y remediarlas	Análisis en el registro; postura sobre lo vivo; flujo de hallazgos con métricas
RV.3	Analizar la causa raíz y mejorar el proceso	Postmortem del Capítulo 7

El marco no exige herramientas: exige que cada práctica tenga un control que la cubra y evidencia de que corrió. Esa es la columna derecha.

ACTIVIDAD EN GRUPO · 25 MINUTOS

Actividad A2 · Políticas y análisis dinámico para su caso

Para su caso de la sesión 1, con el pipeline que diseñaron en la sesión 4 y la cartera de pruebas de la sesión 7:

1 · Tres políticas

- Recurso, condición y efecto (deny o warn), en una línea cada una.
- Al menos una sobre exposición y una sobre identidad.
- En qué momento corre cada una: código, plan o lo vivo.

2 · El análisis dinámico

- Dónde entra en su pipeline: etapa, modo y umbral.
- Qué necesita para correr: dirección, especificación, usuario de prueba.
- Qué hace con el hallazgo alto: detiene qué.

3 · El dueño

- Quién escribe las políticas y quién las aprueba.
- Quién autoriza una excepción y por cuánto tiempo.
- Qué cambia respecto de cómo el caso aprueba hoy.

Entregable verbal: cada grupo presenta en dos minutos su política más importante y la ubicación del análisis dinámico. Sin material impreso ni aula virtual.

PRESENTACIÓN · JUEVES 12/11 · 16:00

Laboratorio 5

Pipeline con validación de seguridad de componentes y de código de infraestructura.

Punto de partida

- Los dos pipelines del Laboratorio 3: aplicación con cuatro compuertas e infraestructura con plan comentado.
- Los jobs de dependencias y estático informaban; la política de infraestructura corría solo sobre el código declarado.

Qué se agrega

- Composición como compuerta con umbral escrito, inventario publicado con el digest y excepción con cuatro datos.
- Análisis de la imagen antes del push; acciones y base fijadas por digest.
- Política propia sobre el plan de infraestructura, con deny y warn, y excepción pegada al recurso.

La guía se publica en el aula virtual una semana antes de la sesión de laboratorio.

Puntos clave

1. **La configuración insegura es una vulnerabilidad sin código**
   Se declara como código y se revisa como código: en el commit, en el plan y sobre lo vivo.

2. **La política se escribe, se versiona y se evalúa en el pipeline**
   deny detiene, warn informa, y la excepción vive junto al recurso con vencimiento.

3. **El análisis dinámico corre contra certificación con la dirección del despliegue**
   Pasivo en cada despliegue, activo por reloj; su evidencia es la petición y la respuesta.

4. **Diez controles caben en una página con etapa, efecto, umbral y evidencia**
   Esa página es la política de seguridad del pipeline.

5. **Un solo flujo de hallazgos, en la herramienta del equipo**
   Triage con criterio escrito, decisión registrada y tres métricas.

CIERRE SESIÓN 10

Para la sesión 11

Preguntas para cerrar

- De los diez controles del mapa, ¿cuáles tiene hoy su pipeline del Laboratorio 3 y en qué efecto?
- ¿Qué revisión manual de su caso de la sesión 1 se podría escribir hoy como política?
- Cuando un control detiene en producción a las 2 a. m., ¿quién decide y con qué criterio escrito?

Antes de la sesión 11

- Leer la sección de análisis dinámico de la guía DevSecOps de OWASP (bibliografía complementaria).
- Traer un incidente que conozcan, propio o público, con su causa y cómo se comunicó.
- Laboratorio 5: jueves 12/11, 16:00. La guía se publica el 5/11.

La sesión 11 abre el Capítulo 7 con la pregunta que la tercera fila deja abierta: qué pasa después del incidente, y cómo se aprende de él sin buscar culpables.